OpenShift Platform Blueprints

Guide V5 - Référentiel consolidé OpenShift, GitOps, sécurité, observabilité, multi-cluster et licensing

Nature du document	Polycopié premium / book de consolidation technique
Public cible	Architecte, ingénieur plateforme, admin OpenShift, profil DevOps / SRE, préparation mission ou certification
Angle	Transformer une masse documentaire OpenShift en parcours lisible, structuré et réutilisable
Auteur	Zidane Djamal

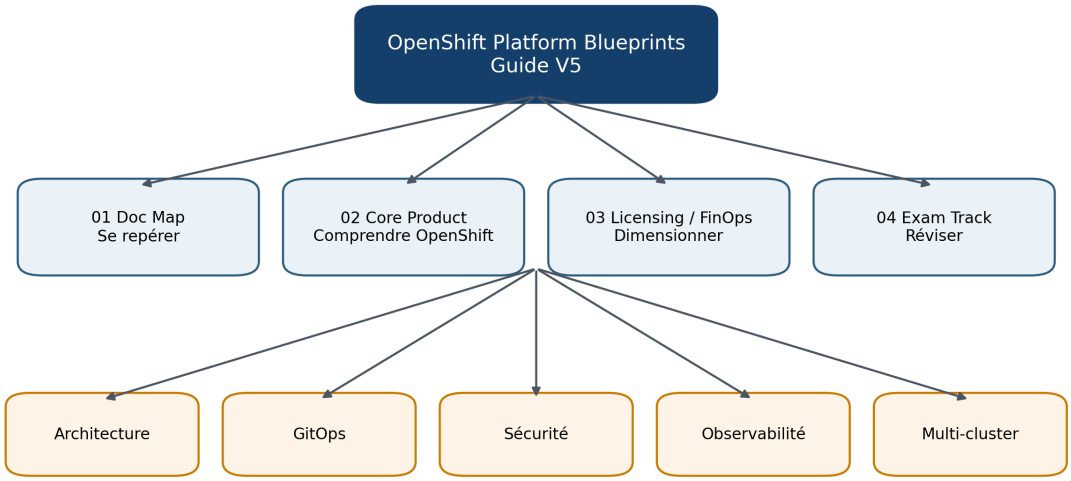


Figure de couverture - lecture d'ensemble du guide et des blocs structurants.

Sommaire

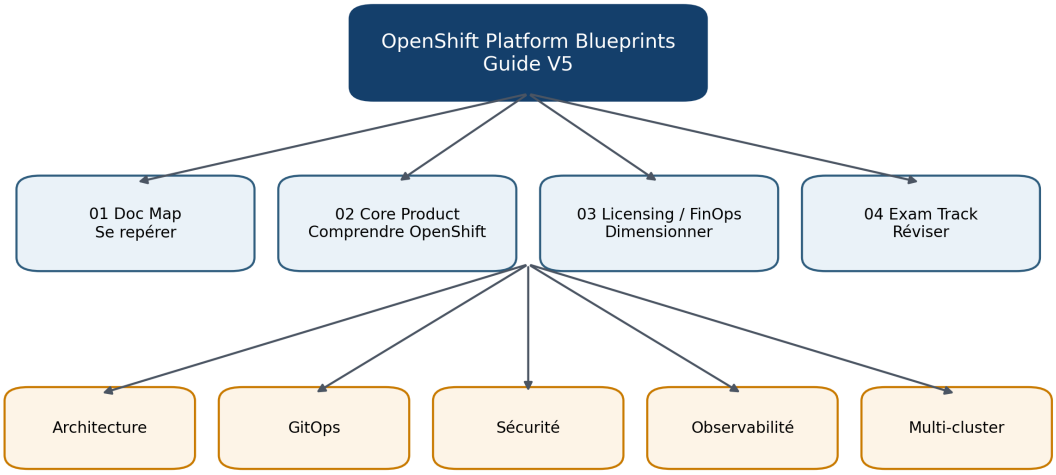
Partie	Contenu
1-5	Comment lire ce guide ; cartographie documentaire ; architecture cluster ; installation ; post-installation
6-10	RBAC ; workloads, services et routes ; registre d'images ; réseau ; stockage
11-15	Sécurité ; authentification et SSO ; observabilité ; Operators ; air gap
16-19	Mise à jour et migration ; licensing / FinOps ; recommandations d'architecture ; feuille de route

Mode d'emploi

Le document se lit comme un livre de consolidation. On peut le parcourir de manière linéaire, ou l'utiliser comme base de référence avant de plonger ensuite dans les blueprints exécutables et les use cases du dépôt.

1. Comment lire ce guide

Ce document regroupe les briques essentielles d'OpenShift dans une logique de plateforme. L'objectif n'est pas de recopier la documentation officielle, mais de produire un support plus compact, plus cohérent et plus utile pour l'architecture, l'exploitation, la préparation de mission et la montée en compétence.



Le guide relie quatre angles de lecture : cartographie documentaire, compréhension produit, architecture de plateforme et cadrage licensing / FinOps.

Ce que le guide cherche à démontrer

Une plateforme OpenShift crédible ne se limite pas à déployer des pods. Elle combine architecture du cluster, modes d'installation, sécurité, réseau, stockage, identités, GitOps, observabilité, opérations de jour 2, gouvernance multi-cluster et logique de licensing.

Parcours recommandé

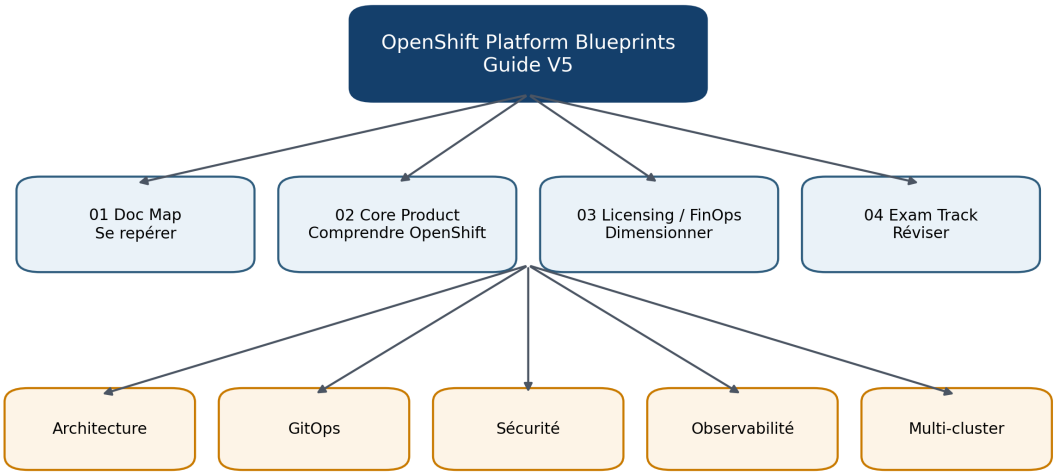
• Commencer par les chapitres 2 à 5 pour comprendre la structure documentaire, l'architecture cluster, l'installation et la post-installation.

• Poursuivre par les chapitres 6 à 15 pour maîtriser le coeur produit : RBAC, workloads, images, réseau, stockage, sécurité, SSO, observabilité et Operators.

• Lire les chapitres 16 à 19 pour traiter les sujets de maturité plateforme : air gap, mise à jour, licensing / FinOps et recommandations d'architecture selon le contexte.

2. Cartographie documentaire OpenShift

La documentation OpenShift est vaste. Une lecture efficace commence par une carte : vue d'ensemble, grands chapitres fonctionnels, puis approfondissement ciblé selon le besoin.



Cartographie simplifiée : doc map, coeur produit, licensing / FinOps et piste exam.

Les fichiers de cartographie montrent que la documentation peut être lue à travers quatre portes d'entrée : la vue d'ensemble, les chapitres fonctionnels, les sujets d'installation / évolution, puis le lien avec les parcours de certification.

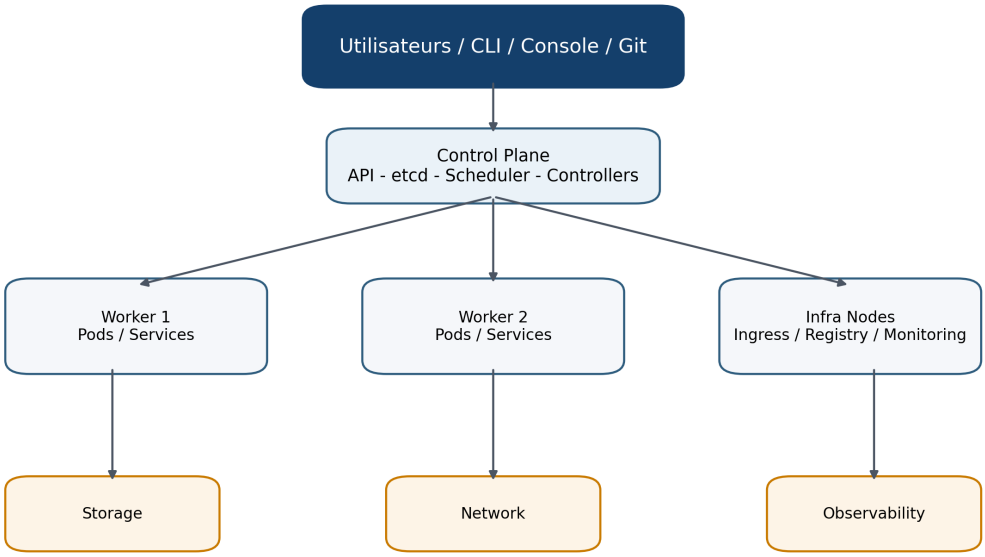
Bloc	Rôle	Exemples de thèmes
01 Doc Map	Orienter la lecture	overview, discover architecture, install, update & migrate, functional chapters, mindmap
02 Core Product	Comprendre le produit OpenShift	RBAC, networking, storage, security, auth, observability, OLM, disconnected
03 Licensing / FinOps	Dimensionner et choisir	éditions, comptage, scénarios, FAQ, arbres de décision
04 Exam Track	Réviser et prioriser	corrélation avec EX280, EX288, EX380, EX480

À retenir

Toujours raisonner par version de cluster. Les concepts fondamentaux restent stables, mais les détails d'implémentation, les opérateurs recommandés et certaines interfaces changent selon les versions.

3. Architecture du cluster OpenShift

OpenShift repose sur Kubernetes, mais ajoute un ensemble dense de composants pour livrer une plateforme de conteneurs complète. La lecture du cluster commence par la distinction entre control plane, workers et éventuels noeuds infra.



Vue simplifiée du cluster : control plane, workers, services d'infrastructure et briques transverses.

Le control plane porte la décision et l'état du cluster : API Server, etcd, scheduler et controllers. Les workers exécutent les charges de travail. Les noeuds infra, lorsqu'ils existent, isolent les services partagés tels que l'ingress router, le registre interne ou certaines briques de monitoring.

Composant	Fonction principale	Pourquoi c'est critique
API Server	Point d'entrée central des requêtes REST	Sans lui, aucune opération de gestion ou de déploiement n'est possible
etcd	Base clé-valeur source de vérité	La perte du quorum etcd compromet le cluster
Scheduler	Assigne les nouveaux pods	Arbitre placement, contraintes et ressources
Controller Manager	Régule l'état désiré	Maintient la convergence des objets
Kubelet	Agent principal du noeud	Un worker sans kubelet sain passe NotReady

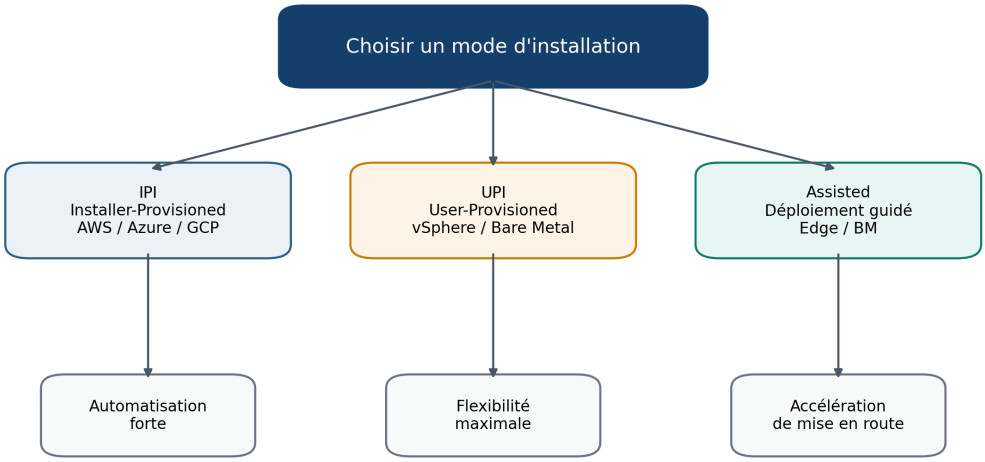
Composant	Fonction principale	Pourquoi c'est critique
CRI-O	Runtime de conteneurs	OpenShift ne repose plus sur Docker pour exécuter les conteneurs

Point d'architecture important

En production, la séparation control plane / workers est la règle. Lorsque la plateforme monte en maturité, on introduit souvent des noeuds infra pour décharger les services partagés et mieux isoler les applications métier.

4. Installation : IPI, UPI et Assisted

L'installation conditionne le niveau d'automatisation, la flexibilité de l'infrastructure et la responsabilité opérationnelle. OpenShift distingue principalement l'IPI, l'UPI et des modes assistés pour certains contextes.



Trois grandes familles : IPI pour l'automatisation, UPI pour la maîtrise fine, Assisted pour accélérer certains déploiements.

Mode	Quand le choisir	Atout principal	Point de vigilance
IPI	Cloud public ou plateforme supportée	Provisionnement automatisé de l'infrastructure	Moins de liberté sur les choix d'infrastructure
UPI	vSphere, bare metal, on-prem, contexte très contraint	Contrôle maximal	Réseau, DNS et bootstrap plus exigeants
Assisted	Parcours guidé, edge ou certains bare metal	Accélère le démarrage	Il faut tout de même maîtriser les prérequis

Quel que soit le mode retenu, les mêmes points restent critiques : DNS, connectivité réseau, dimensionnement minimal, choix de la version, cohérence du fichier install-config.yaml et sauvegarde de ce dernier après déploiement.

5. Post-installation : rendre le cluster exploitable

Un cluster fraîchement installé n'est qu'une base. La post-installation le transforme en plateforme prête à accueillir des applications et des équipes : stockage par défaut, identité, certificats, monitoring, exposition du registre et premières conventions d'exploitation.

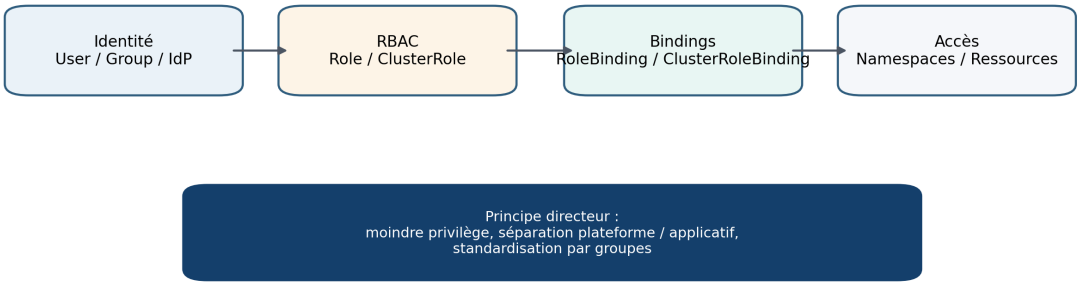
Axe	Décision à prendre	Conséquence si oublié
StorageClass par défaut	Définir la classe de stockage de référence	De nombreux PVC ne se provisionnent pas correctement
IdP / OAuth	Choisir HTPasswd, LDAP, OIDC ou autre	Gestion des identités limitée et peu réaliste
Certificats	Remplacer les autosignés si nécessaire	Chaîne de confiance fragile, mauvaise UX et intégration plus difficile
Monitoring	Ajuster rétention, alertes et tableaux de bord	Visibilité insuffisante ou ressources mal dimensionnées
Registry	Configurer stockage et exposition	Cycle de build et de déploiement pénalisé

Conseil de plateforme

Traiter l'espace openshift-config comme une zone de configuration critique. Ce que l'on place ici doit être versionné, revu et sauvegardé.

6. Projets, namespaces et RBAC

Le multi-tenancy OpenShift se lit d'abord par les projets puis par les permissions. Un projet correspond à un namespace enrichi ; il isole ressources, politiques et comptes de service. Le RBAC organise ensuite qui peut faire quoi et à quel niveau.



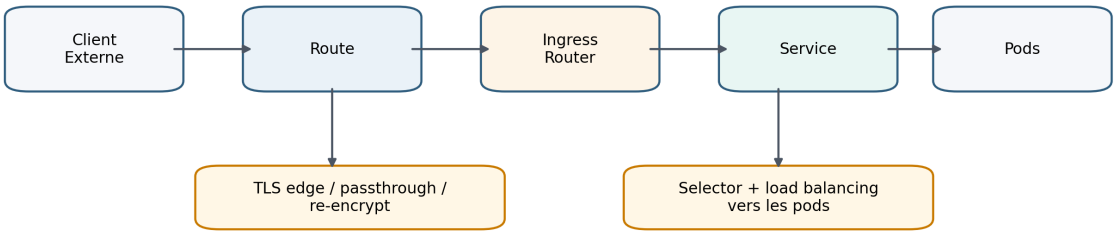
Le flux d'autorisation combine identité, rôles, bindings et portée d'accès sur les espaces et les ressources.

OpenShift fournit des rôles par défaut très utiles - cluster-admin, admin, edit et view - mais la bonne pratique reste le moindre privilège. Un rôle namespaced répond à un besoin local ; un clusterrole intervient lorsqu'il faut agir à l'échelle du cluster ou sur plusieurs namespaces.

Objet	Portée	Usage typique
Role	Un projet	Autoriser la gestion des deployments, services et routes dans un namespace
ClusterRole	Cluster ou tous les projets	Lire les nodes, administrer certaines ressources globales
RoleBinding	Un projet	Attacher un role à un utilisateur, groupe ou service account local
ClusterRoleBinding	Cluster	Accorder un clusterrole à un sujet à l'échelle globale

7. Workloads, Services et Routes

Déployer sur OpenShift consiste à choisir le bon type de workload, à lui donner un point d'accès stable via un Service, puis à l'exposer si nécessaire au travers d'une Route.



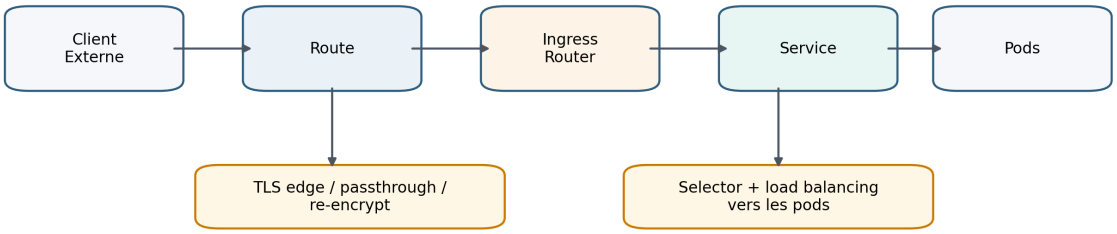
Chaîne d'exposition standard : client, route, ingress router, service, pods.

Workload	Quand l'utiliser	Point d'attention
Deployment	Applications stateless	Stratégie standard recommandée pour les nouveaux workloads
StatefulSet	Bases de données, identités stables, stockage persistant	Exige une stratégie stockage et sauvegarde propre
DaemonSet	Agent par noeud	Utile pour collecte de logs ou monitoring
Job / CronJob	Traitements batch et planifiés	Bien gérer la répétition, la rétention et les ressources

OpenShift conserve aussi DeploymentConfig, mais Deployment est aujourd'hui le socle recommandé. Pour l'exposition externe, la Route est l'objet spécifique OpenShift ; elle permet notamment les terminaisons TLS edge, passthrough et re-encrypt.

8. Registre d'images, ImageStreams et builds

OpenShift simplifie le cycle de vie des images grâce au registre interne, aux ImageStreams et à plusieurs stratégies de build. Cette couche est centrale pour relier code source, pipelines et déploiements.



Le chemin d'exposition applicative dépend souvent en amont d'un cycle de build et de push d'image maîtrisé.

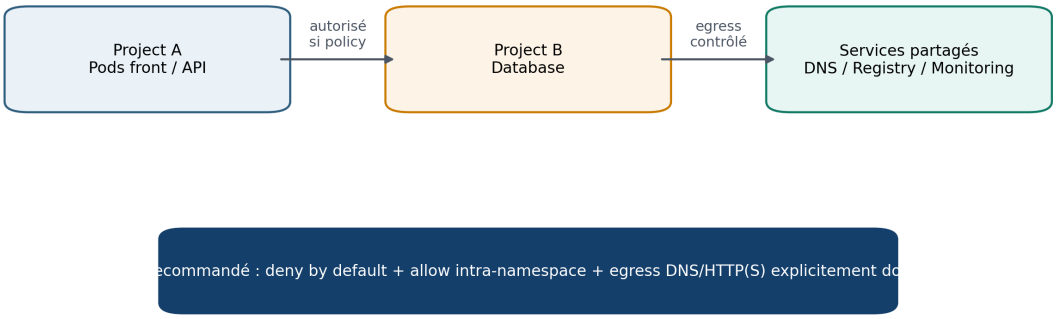
Brique	Rôle	Valeur pratique
Registre intégré	Stocker les images produites ou poussées	Point central de distribution avec contrôle RBAC
ImageStream	Abstraction par tags symboliques	Découple les déploiements des digests exacts et facilite les triggers
S2I	Build sans Dockerfile	Standardise et accélère l'industrialisation de certains stacks applicatifs
Build Docker	Build à partir d'un Dockerfile	Flexibilité maximale
Pipeline	Chaîne CI/CD plus complète	Ajoute tests, scans et orchestration

Point d'architecture

Le registre intégré a besoin d'un backend de stockage fiable. Sans stratégie de stockage claire, toute la chaîne build / image / déploiement devient fragile.

9. Réseau, Services et politiques réseau

Le réseau OpenShift ne se limite pas à exposer des services. Il organise les communications pod à pod, service à pod, cluster vers extérieur, ainsi que l'isolation entre projets.



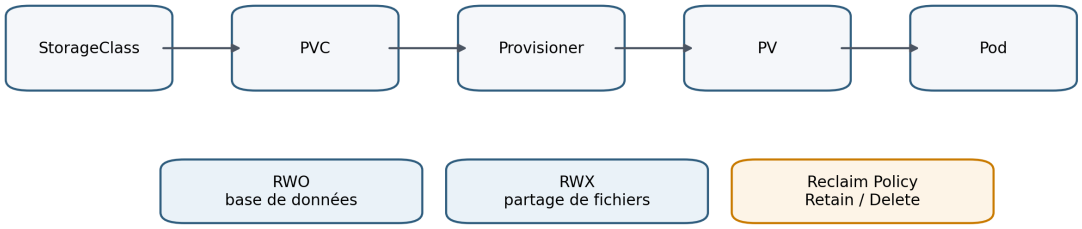
Le pattern recommandé reste : segmentation logique, service discovery interne et policies explicites.

Chaque pod reçoit une IP. Les Services apportent une adresse stable et kube-proxy réalise le load balancing. L'Ingress Controller traite l'entrée du trafic externe à travers les Routes. Les NetworkPolicies servent ensuite à formaliser les flux permis.

Sujet	À comprendre	Piège fréquent
Pod-to-Pod	La communication inter-noeuds via le réseau overlay	Supposer qu'une policy existe alors qu'aucune n'est définie
Service	L'adresse stable et la sélection par labels	Mauvais selector, mauvais port ou targetPort
Route	L'exposition externe et la terminaison TLS	Exposer trop vite sans politique de sécurité claire
NetworkPolicy	Pare-feu des pods	Oublier le basculement implicite vers deny-all sur les pods sélectionnés

10. Stockage persistant

Le stockage dans OpenShift repose sur une séparation nette des responsabilités : l'administrateur offre des classes de stockage et les développeurs réclament du stockage via des PVC.



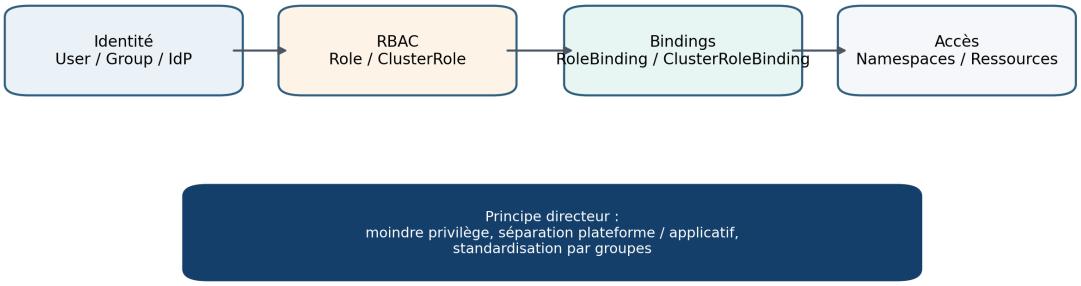
Chaîne de vie du stockage : StorageClass, PVC, provisioner, PV, pod.

Objet	Porté par	Rôle
StorageClass	Administrateur	Décrire les classes de service disponibles et le provisioner associé
PersistentVolume	Cluster / provisioner	Volume concret alloué
PersistentVolumeClaim	Développeur / workload	Demande de capacité, mode d'accès et classe

Les modes d'accès RWO, RWX et ROX ne sont pas interchangeables. Le choix du stockage dépend du cas d'usage : base de données, partage de fichiers, données statiques, réplication, stratégie de sauvegarde et reclaim policy.

11. Sécurité : SCC, secrets, conformité

OpenShift adopte une sécurité en couches. Les Security Context Constraints, les standards de sécurité des pods, les secrets et les opérateurs de conformité matérialisent cette approche.



Le contrôle d'accès et la sécurité d'exécution doivent être pensés ensemble.

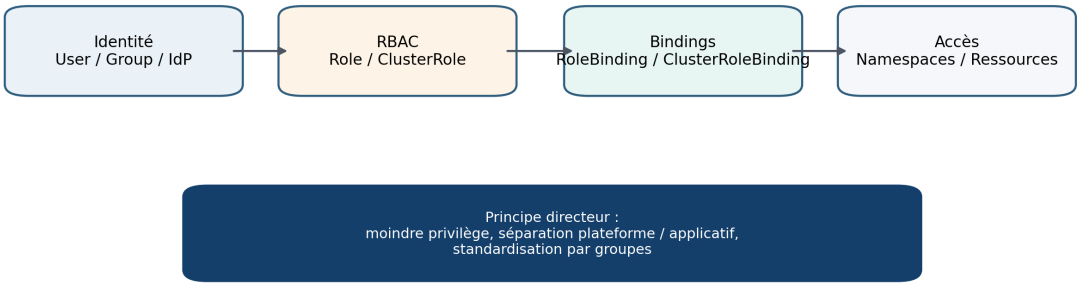
Brique	Ce qu'elle contrôle	À retenir
SCC	Privilèges d'exécution, hostPath, capabilities, UID	restricted-v2 est la norme et peut surprendre les images conçues pour root
Pod Security Standards	Niveaux privileged / baseline / restricted	Renforce la normalisation côté Kubernetes
Secrets	Données sensibles	À séparer strictement des ConfigMaps et à protéger dans etcd
Compliance Operator	Scans et remédiations de conformité	Aide à la conformité, ne remplace pas la compréhension des concepts

Bonne pratique

Activer le chiffrement au repos pour etcd et éviter toute donnée sensible dans un ConfigMap. C'est un fondamental de sécurité plateforme.

12. Authentification, OAuth et SSO

OpenShift délègue l'authentification à son opérateur d'authentification de cluster et peut s'intégrer à plusieurs IdP : HTPasswd, LDAP, OIDC, GitHub, GitLab ou d'autres fournisseurs compatibles.



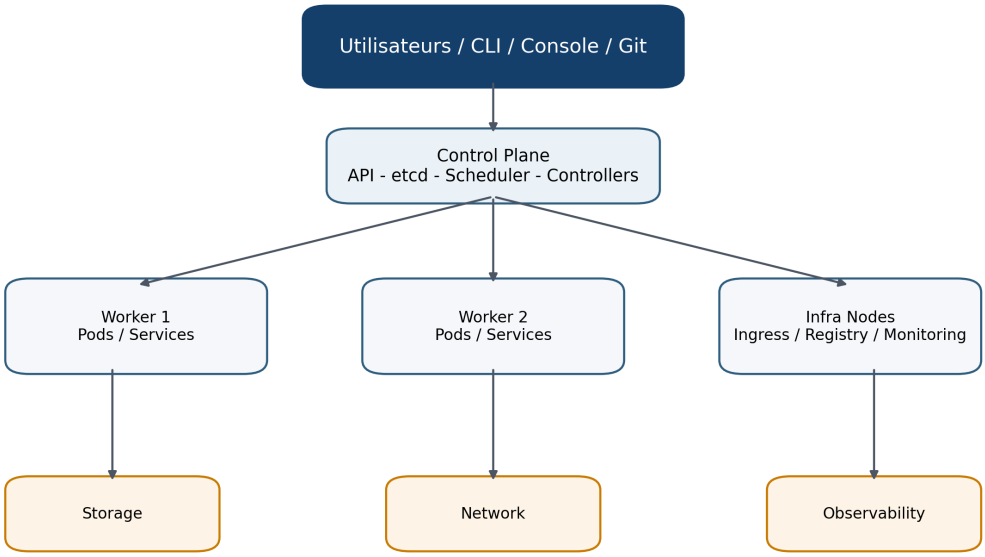
Le SSO s'intègre à l'identité ; le RBAC reste responsable de l'autorisation.

IdP	Usage	Comment le positionner
HTPasswd	Tests, labs, accès de secours	Simple mais non recommandé en production
LDAP / AD	Annuaire d'entreprise	Bon choix pour centraliser les utilisateurs et groupes
OIDC	SSO moderne	Intègre Keycloak, Okta, Azure AD et autres

L'authentification n'est pas l'autorisation. Un utilisateur authentifié n'a pas automatiquement les droits ; les rôles et bindings déterminent ensuite ce qu'il peut faire dans le cluster.

13. Observabilité, opérations et jour 2

Une plateforme OpenShift crédible doit être observable. Le monitoring, les logs, les alertes et les opérations de jour 2 constituent le passage vers une vraie lecture SRE de la plateforme.

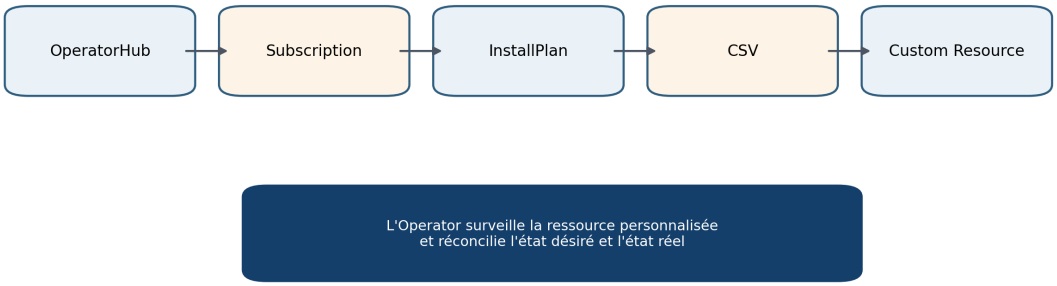


Les briques d'observabilité et d'opérations gravitent autour du cluster, des services partagés et des workloads.

Brique	Fonction	Question d'architecture
Prometheus	Collecte de métriques	Quelle rétention, quels scrape targets, quelle consommation ressources ?
Alertmanager	Diffusion des alertes	Comment éviter le bruit et garder des alertes actionnables ?
Logging	Centraliser les logs	Quel backend : Loki, ELK, autre ? quel coût ?
Machine API	Cycle de vie des noeuds	Disponible surtout dans les contextes IPI supportés
Sauvegarde etcd / OADP	Protection de la plateforme et des applications	Quelle stratégie de backup, restauration et test de reprise ?

14. Operators et OLM

Les Operators encapsulent une connaissance d'exploitation dans du logiciel. OLM gère leur installation, leurs dépendances et leurs mises à jour via OperatorHub, Subscription, InstallPlan et CSV.



Cycle de vie OLM simplifié : choix de l'Operator, souscription, plan d'installation, CSV puis ressource personnalisée.

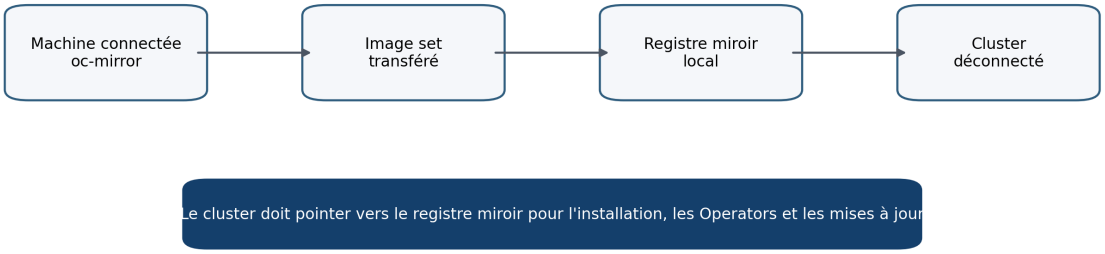
Le pattern Operator devient déterminant dès que l'on déploie des composants complexes : base de données, plateforme de logs, stockage, sécurité, observabilité ou middleware. Il rapproche l'administration d'un service de la logique d'un service cloud managé.

Réflexe utile

Avant de déployer manuellement un composant complexe, vérifier s'il existe un Operator viable et supporté. Cela influence directement l'exploitabilité du service dans le temps.

15. Installations déconnectées et air gap

Le mode déconnecté impose de maîtriser le cycle complet de mise en miroir : images de release, opérateurs, registres privés, transfert de contenu et paramétrage de l'installation pour pointer vers ce miroir.



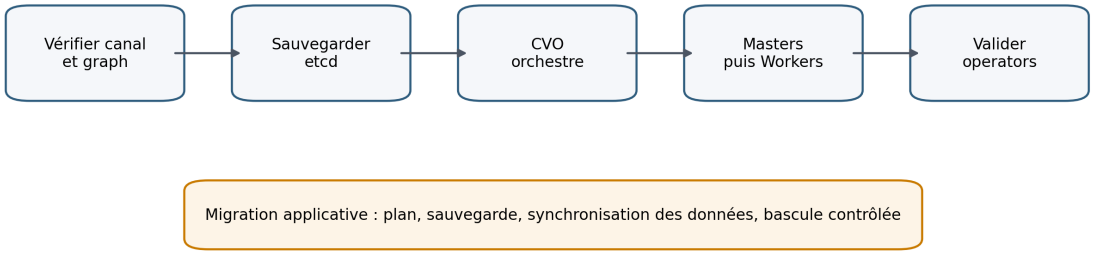
Flux de base : oc-mirror sur machine connectée, transfert, publication sur registre local, puis consommation par le cluster.

Le registre miroir est la pièce maîtresse du dispositif. Tous les noeuds doivent y accéder. Le volume de données à mirrorer peut devenir important, en particulier lorsque l'on ajoute plusieurs versions, Operators et dépendances.

Étape	But	Point de vigilance
Créer l'image set	Sélectionner le contenu OpenShift / Operators	Bien gérer versions et taille du miroir
Transférer	Acheminer le contenu vers la zone air gap	Capacité de stockage et logistique
Publier sur le registre local	Rendre les images disponibles au cluster	Disponibilité, certificats, accès
Configurer install-config	Pointer l'installation vers le miroir	Cohérence des sources et du pull secret

16. Mise à jour et migration

Mettre à jour OpenShift est un processus orchestré par le Cluster Version Operator. Migrer, en revanche, implique souvent bien plus qu'un simple upgrade : applications, données, stratégies de bascule et outils comme MTC.



Upgrade : vérifier le canal, sauvegarder etcd, laisser le CVO orchestrer, puis valider la santé du cluster.

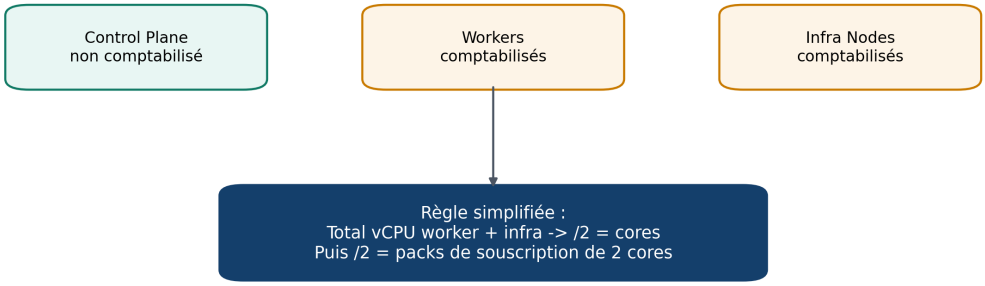
Le graphe de mise à jour interdit les sauts arbitraires de versions mineures. Avant toute mise à jour sérieuse, la sauvegarde d'etcd et la lecture des notes de version sont incontournables. En migration, la difficulté principale réside dans la bascule des données et la gestion du downtime.

Disciplines de base

Toujours tester la procédure sur un environnement non productif, conserver un point de retour clair et valider séparément control plane, workers et operators après la mise à jour.

17. Licensing et FinOps OpenShift

Le licensing OpenShift influence fortement le design de plateforme. Il faut distinguer ce qui est comptabilisé, choisir la bonne édition et anticiper la croissance du cluster dès la phase d'architecture.



Lecture simplifiée du comptage : control plane non comptabilisé, workers et infra nodes comptabilisés.

Édition	Positionnement	Quand la choisir
OKE	Base Kubernetes robuste et supportée	Quand on n'a pas besoin de la plateforme complète
OCP	Edition standard la plus courante	La plupart des cas d'usage en production
OCPP	OCP + ACM + ACS + Quay	Plusieurs clusters, exigences sécurité et gouvernance élevées

Règle de comptage	Interprétation
Workers	Toujours comptabilisés car ils exécutent les workloads utilisateur
Infra nodes	Comptabilisés également s'ils portent routeurs, registre, monitoring ou autres composants infra
Control plane	Non comptabilisé dans la règle simplifiée présentée dans les fichiers
Conversion	2 vCPU = 1 core ; les souscriptions sont généralement vendues par packs de 2 cores

Lecture FinOps

Le licensing n'est pas un appendice commercial. Il agit sur le design : nombre de noeuds infra, stratégie multi-cluster, usage des offres managées, et niveau d'automatisation de la capacité.

18. Recommandations d'architecture par contexte

Le bon design OpenShift dépend moins d'un modèle universel que du contexte. Ce chapitre propose des orientations structurantes selon quatre situations récurrentes : lab, plateforme interne, production régulée et trajectoire multi-cluster.

Contexte	Architecture cible	Priorités	À éviter
Lab / apprentissage	1 cluster simple, identité minimale, quelques workloads de démo	Lisibilité, compréhension produit, premiers patterns	Complexité prématurée et promesses de production
Plateforme interne standard	OCP, GitOps, monitoring, RBAC propre, storage défini	Standardisation, time to platform, relecture Git	Administration manuelle dispersée
Production régulée	Sécurité renforcée, traçabilité, sauvegardes testées, IdP d'entreprise	Conformité, durcissement, réseau, secrets, air gap éventuel	HTPasswd durable, secrets mal gérés, absence de backup
Trajectoire multi-cluster	Cluster management + clusters workloads, GitOps étendu	Gouvernance, visibilité globale, cohérence des politiques	Multiplier les clusters sans raison lisible

Dans les contextes cloud managés comme ROSA, ARO ou ROKS, une partie du control plane et de l'infrastructure est déléguée au fournisseur, ce qui réduit la charge opérationnelle mais modifie la lecture des coûts et la responsabilité de certains composants.

19. Conclusion et feuille de route de consolidation

Le guide V5 doit être lu comme un support de cadrage, pas comme un terminal absolu. Il transforme une documentation éparse en un parcours cohérent qui aide à comprendre OpenShift, à l'architecturer, à l'exploiter et à le présenter dans une logique portfolio ou mission.

La prochaine étape logique consiste à convertir cette base documentaire en deux artefacts complémentaires : d'une part un livre de référence produit comme celui-ci ; d'autre part des blueprints exécutables dans platform/ avec YAML fiabilisés, schémas GitOps cohérents et use cases phares visibles.

Feuille de route recommandée
1) Stabiliser les blueprints exécutables ; 2) mettre en avant deux ou trois use cases majeurs ; 3) aligner architecture, docs et artifacts GitOps ; 4) préparer une version orientée portfolio / recruteur ; 5) conserver ce guide comme référentiel de fond.

Auteur : Zidane Djamal - Architecte technique / plateforme / cloud-native